

JavaScript & the DOM

COMPLETE LEARNING GUIDE

What You'll Learn

Before any framework, the browser gives you a live, mutable tree representation of the page — the DOM — and a JavaScript API for reading and changing it. Understanding this layer directly is what makes React's abstractions (JSX, synthetic events, the virtual DOM diffing it does on your behalf) make sense rather than feeling like magic, and it's what you fall back on for the parts frameworks don't cover, like measuring element sizes or integrating a third-party widget. This guide covers selecting and traversing elements, creating and modifying nodes, the event system including bubbling/capturing and delegation, form handling, fetch-based async updates, and the debounce/throttle patterns that keep DOM-heavy interactions performant. These are the exact mechanics React re-implements under the hood.

1. Selecting and Traversing the DOM

`document.querySelector` and `document.querySelectorAll` accept any CSS selector and are the modern default for finding elements, replacing older single-purpose methods like `getElementById` and `getElementsByClassName`. `querySelector` returns the first match (or null); `querySelectorAll` returns a static `NodeList` — a snapshot at call time, not a live view, meaning it won't automatically include elements added to the DOM afterward. Once you have a reference, `.parentElement`, `.children`, `.nextElementSibling`, and `.closest(selector)` let you navigate relative to it without re-querying the whole document, which matters both for performance and for correctness inside event handlers where you often need 'the row this button lives in' rather than the button itself.

- `querySelector('.card')`/`querySelectorAll('.card')` work with any valid CSS selector, including combinators like `'.list > li:first-child'`.
- `element.closest('.card')` walks up the ancestor chain looking for a match — the standard way to find a containing component from a deeply nested click target.
- `NodeList` from `querySelectorAll` isn't an `Array` — use `Array.from(list)` or `[...list]` before calling `.map()`/`.filter()` on it.

2. Creating, Modifying, and Removing Elements

`document.createElement` builds a new, detached node; setting its properties (`.textContent`, `.className`, `.dataset`) and then calling `appendChild` or the more flexible `append/prepend/before/after` inserts it into the tree. `.textContent` is the safe way to set text content — it never parses HTML, so user-supplied text can't inject markup. `.innerHTML` parses and executes its string as HTML, which is powerful for templating but a direct cross-site-scripting vector if the string includes any untrusted user input. Removing nodes is done with

`element.remove()` (modern) rather than the older `parent.removeChild(element)` pattern, though both still work.

JAVASCRIPT

```
function renderStudentCard(student) {
  const card = document.createElement("div");
  card.className = "student-card";
  card.dataset.studentId = student.id;

  const name = document.createElement("h3");
  name.textContent = student.name; // safe: never parsed as HTML
  card.append(name);

  return card;
}

const list = document.querySelector("#student-list");
students.forEach((s) => list.append(renderStudentCard(s)));
```

3. Event Handling and the Event Object

`addEventListener(type, handler)` attaches a listener without overwriting any existing ones, unlike setting `element.onclick` directly, which allows only a single handler. The event object passed to the handler carries useful context: `event.target` is the exact element that triggered the event, `event.currentTarget` is the element the listener is attached to (these differ during bubbling), and `event.preventDefault()` stops default browser behavior (like a form submitting or a link navigating) without stopping the event from continuing to propagate. Removing a listener later requires `removeEventListener` with a reference to the exact same function — an inline arrow function passed directly can never be removed because a new function reference is created each time.

JAVASCRIPT

```
function handleClick(event) {
  console.log("clicked:", event.target);
}

const btn = document.querySelector("#save-btn");
btn.addEventListener("click", handleClick);

// Later, this only works because it's the SAME function reference:
btn.removeEventListener("click", handleClick);
```

4. Event Delegation, Bubbling, and Capturing

Most DOM events bubble: they fire on the target element first, then travel up through each ancestor. Event delegation exploits this by attaching a single listener to a stable parent container instead of one listener per child, then inspecting `event.target` inside the handler to determine which child was actually interacted with. This is significantly more efficient for lists that render many items or change dynamically, since you attach one listener instead of re-attaching listeners every time the list updates. Capturing (the less common alternative, opted into with `{ capture: true }`) fires listeners top-down before bubbling begins, and is mostly used

for intercepting events before a child component can handle them.

- Delegating a click listener to a `<ul>` and checking `event.target.closest('li')` handles clicks on any current or future `<li>`, including ones added after the listener was attached.
- `event.stopPropagation()` halts bubbling entirely — reach for it deliberately and rarely, since it can silently break delegation set up higher in the tree.

5. Forms, Validation, and preventDefault

A form's default behavior is a full-page navigation/reload on submit — almost never what a modern interactive page wants, so listening for the 'submit' event and calling `event.preventDefault()` is close to mandatory. HTML5 gives you built-in validation attributes (required, pattern, minlength, type='email') that the browser enforces automatically and surfaces with `:invalid/:valid` CSS pseudo-classes, but any validation that depends on server-side state (like checking whether a username is already taken) still needs JavaScript. `FormData(formElement)` is the standard way to read all field values at once, including files, without manually querying every input.

JAVASCRIPT

```
document.querySelector("#signup-form").addEventListener("submit", (event) => {
  event.preventDefault();
  const data = new FormData(event.target);
  const payload = Object.fromEntries(data.entries());
  submitSignup(payload);
});
```

6. Async JavaScript: fetch, Promises, and Updating the DOM

`fetch(url)` returns a Promise that resolves once the HTTP response headers arrive — the body still needs to be read asynchronously via `.json()` or `.text()`, which itself returns another Promise, which is why a typical fetch call chains two awaits (or two `.then()` calls). `async/await` is syntactic sugar over Promises that lets asynchronous code read top-to-bottom like synchronous code, and `try/catch` around an awaited fetch is the standard way to handle both network failures and non-2xx responses (fetch, notably, does not throw on a 404 or 500 — you must check `response.ok` yourself).

JAVASCRIPT

```
async function loadStudents() {
  try {
    const response = await fetch("/api/students");
    if (!response.ok) throw new Error(`HTTP ${response.status}`);
    const students = await response.json();
    renderStudentList(students);
  } catch (err) {
    showErrorBanner("Could not load students.");
  }
}
```

7. Debouncing and Throttling for Performance

Events like scroll, resize, and keyup on a search input can fire dozens of times per second, and running expensive work (a DOM re-render, an API call) on every single firing will visibly lag the page. Debouncing delays execution until a pause of a given length has occurred since the last call — ideal for search-as-you-type, where you only want to fire the request once the user stops typing. Throttling instead guarantees execution happens at most once per interval regardless of how often the event fires — better suited to scroll-position tracking, where you want regular updates but not on every single pixel of movement.

JAVASCRIPT

```
function debounce(fn, delayMs) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delayMs);
  };
}

const debouncedSearch = debounce((query) => fetchResults(query), 300);
searchInput.addEventListener("input", (e) => debouncedSearch(e.target.value));
```

Common Pitfalls

- Setting `.innerHTML` with any string that includes user-supplied input, opening a direct XSS vulnerability instead of using `.textContent`.
- Attaching a new event listener every time a list re-renders instead of using event delegation, leaking listeners over time.
- Trying to `removeEventListener` with a freshly created inline arrow function, which can never match the originally attached reference.
- Assuming `fetch()` rejects on HTTP error status codes — it only rejects on network failure, so `response.ok` must be checked explicitly.
- Querying the same DOM element repeatedly inside a loop or handler instead of caching the reference once.
- Confusing `event.target` (where the event originated) with `event.currentTarget` (where the listener is attached) during bubbling.
- Running expensive logic directly on scroll/resize/keyup without debouncing or throttling, causing visible jank.

Key Takeaways

- `querySelector/querySelectorAll` with CSS selectors are the modern default for DOM lookups.
- `textContent` is safe by default; `innerHTML` executes markup and is a security decision, not just a convenience choice.

- Event delegation (one listener on a parent, inspect event.target) scales far better than per-item listeners.
- fetch requires checking response.ok manually — it does not throw for HTTP error statuses.
- Debounce for 'wait until they stop'; throttle for 'no more than once every X ms' — picking the wrong one causes either laggy or overly-chatty UIs.
- Everything a framework like React does — diffing, event handling, updates — is built on exactly these DOM primitives.

Learn More

MDN — Introduction to Events

https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting/Events

MDN — Using Fetch

https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

javascript.info — Bubbling and Capturing

<https://javascript.info/bubbling-and-capturing>